פיתון (עמודים 1-4)

בסיסי נתונים (עמודים 5-6)

מבנה שאלון הבחינה כמו בשנה שעברה .

פיתון

יש ללמד לפי כל התכנים המופיעים בתוכנית הלימודים ולתרגל בחינות משנים קודמות.

נושאים ותכנים חשובים שיש לשים עליהם דגש ולתרגל בפיתון :

- אלגוריתמיקה של פירוק מספר שלם לספרותיו, בדגש על פעולות // (חילוק בשלמים), % (שארית חלוקה)
- פעולת חזקה (**) עם בסיס 10
- לולאות while,for , לולאות מקוננות , שימוש בזקיף (ערך דמי לסיום הלולאה)
- בדיקת רצפים (איברים סמוכים) ברשימה.
- רשימת מונים ואבחנה בין מיקום האיבר ברשימה לעומת ערך האיבר .
- טיפוס הנתונים מילון (dict):
  - אתחול מילון ריק
  - סריקת מילון באמצעות מפתח (keys), זוג סדור (items)
  - גישה לערך על פי מפתח
  - הכנסת איבר (מפתח-ערך) למילון
- טיפוס הנתונים קובץ (file)
  - קובץ סוג txt בלבד
  - קריאה מקובץ
  - כתיבה לקובץ
  - סריקה של כל שורה בקובץ באמצעות לולאה
  - פירוק שורה למרכיבים באמצעות מפריד ידוע מראש ושימוש ב- split

סיכום ודוגמאות לפונקציות ופעולות בפיתון : מחרוזות וקבצים

קבצים

כשעובדים עם קבצים בפייתון, ישנם 3 שלבים קבועים:

פתיחה ← פעולה (קריאה/כתיבה) ← סגירה.

- **`open(file_name, mode)`**: פתיחת קובץ.
  - `"r"` (Read) - לקריאה בלבד (אם הקובץ לא קיים, תהיה שגיאה).
  - `"w"` (Write) –לכתיבה (מוחק את מה שהיה בקובץ וכותב מחדש, או בונה קובץ חדש).
  - `"a"` (Append) - להוספה (מוסיף טקסט לסוף תוכן הקובץ הקיים בלי למחוק).

- **`file.close()`**: סגירת הקובץ. חובה בכל סוף טיפול בקובץ כדי לשמור את השינויים ולשחרר זיכרון.

- **`file.read()`**: קורא את **כל** הקובץ בבת אחת ומחזיר אותו כפי שהוא, כמחרוזת (טקסט) אחת ארוכה.

- **`file.readline()`**: קורא **שורה אחת בלבד** (בכל פעם שמפעילים אותה מחדש, היא תעבור לשורה הבאה).

- **`file.readlines ()`** או **`list(file)`**: קורא את כל הקובץ ומחזיר **רשימה של מחרוזות**, שבה כל איבר הוא שורה נפרדת מהקובץ (כולל ה-`n\` בסוף השורה).

- **`file.write(text)`**: כותב מחרוזת טקסט לתוך הקובץ (עובד רק במצבי `"w"` או `"a"`).

פונקציות ופעולות על מחרוזות (Strings)

מחרוזות הן טקסט (מיוצגות במרכאות). להלן הפעולות הנפוצות בעבודה עם מחרוזות:

- **`len(text)`**: מחזירה את **אורך המחרוזת** (כמה תווים יש בה, כולל רווחים).

- **`text.split(separator)`**: מפרקת את המחרוזת לפי סימן מסוים (separator) ומחזירה **רשימה** של כל החלקים הללו. אם לא שמים סימן בסוגריים `text.split() ,`
היא מפרקת אוטומטית לפי רווחים.

לדוגמא : מוגדרת מחרוזת של רשימת פירות כשביניהם הסימן - :

```
text = "apple-banana-orange-mango"
result = text.split("-")
print(result)
```

הרשימה result
תכיל 4 איברים נפרדים: `['apple', 'banana', 'orange', 'mango']`

`result[0]` יהיה `'apple'`
`result[1]` יהיה `'banana'`
וכן הלאה.

המשך פונקציות ופעולות על מחרוזות (Strings)

- **`text.strip()`**: מנקה ומורידה רווחים מיותרים (או ירידות שורה כמו `\n`) מתחילת המחרוזת ומסופה. חשוב מאוד כשקוראים מקובץ!
- **`text.lower()` / `text.upper()`** : הופכת את כל האותיות באנגלית לגדולות (Capital) או קטנות.
- **`text.replace(old, new)`**: מחליפה קטע טקסט ישן (`old`) בקטע טקסט חדש (`new`).

למשל:

```
text="אני אוהב לאכול תפוחים"
      # נחליף את המילה "תפוחים" במילה "בננות"
new_text = text.replace("תפוחים", "בננות")

print(new_text)  # יודפס: אני אוהב לאכול בננות
```

- **`text.find(sub)`**: מחפשת קטע טקסט בתוך המחרוזת ומחזירה את **האינדקס (המיקום)** שבו הוא מתחיל. אם הוא לא נמצא, מחזירה 1-.
- **גישה לפי אינדקס וחיתוך (Slicing)**:
  - `text[0]` - התו הראשון במחרוזת text.
  - `text[-1]` - התו האחרון במחרוזת text .
  - `text[1:4]` - חיתוך: מתקבלת תת מחרוזת מהתו במיקום 1 ב text ועד מיקום 4 (לא כולל מיקום 4).

להלן דוגמא לתרגיל קצר וממוקד המשלב קריאת קובץ, שימוש ב split והצגת נתונים כנדרש:

תיאור המשימה

נתון קובץ טקסט של ציוני תלמידים בשם **grades.txt**.

בכל שורה בקובץ מופיעים פרטים של תלמיד בפורמט הבא (מופרדים על ידי הסימן #):
`שם_התלמיד#מקצוע#ציון`

כך נראה קובץ ה text :

```
דני#מתמטיקה#95
דנה#אנגלית#82
רון#מתמטיקה#45
שירה#היסטוריה#88
```

כתבו קטע קוד בפייתון שקורא את הקובץ **grades.txt** ומדפיס **רק את השמות של התלמידים שנכשלו במתמטיקה** (כלומר, המקצוע הוא "מתמטיקה" והציון שלהם נמוך מ-56).

הערות:

1. לא לשכוח לפתוח את הקובץ במצב המתאים במצב קריאה (Read),(`"r"`) ולסגור אותו בסוף (`close`).
2. יש להשתמש ב-`split("#")` כדי להפריד את השורה לשלושת החלקים שלה.
3. **שימו לב:** הציון מגיע מהקובץ כטקסט (מחרוזת).
כדי להשוות אותו למספר (קטן מ-55), יש להמיר אותו למספר שלם בעזרת הפונקציה `int()` .

דוגמא לפתרון – כולל הערות להבהרה - בגוף התוכנית :

```
# 1. פתיחת הקובץ לקריאה בלבד
file = open("grades.txt", "r")

# 2. המרת השורות של הקובץ לרשימה
lines = list(file)

# 3. מעבר על כל שורה ושורה ברשימה
for line in lines:
    # פירוק השורה לפי הסימן # לרשימה של 3 איברים
    data = line.split("#")

    # שליפת הנתונים לפי המיקום שלהם ברשימה החדשה:
    name = data[0]       # השם נמצא באינדקס 0
    subject = data[1]    # המקצוע נמצא באינדקס 1
    # המרת הציון (אינדקס 2) למספר שלם כדי שנוכל להשוות אותו מתמטית
    grade = int(data[2])

    # 4. הבדיקה: האם המקצוע הוא מתמטיקה והציון נמוך מ-56?
    if subject == "מתמטיקה" and grade < 55:
        print(name)      # הדפסת השם של התלמיד שנכשל

# 5. סגירת הקובץ - חובה בסיום העבודה!
file.close()
```

- **המרת טיפוסים (`int`):** נתונים שנקראים מקובץ טקסט תמיד מגיעים כטקסט (`string`). אם תנסו לעשות `data[2]<56` ללא המרה ל- `int`, תתקבל שגיאה (Error) כי אי אפשר להשוות טקסט למספר.
- **ההבדל בין אינדקסים:** ה-`split` יצר לנו רשימה data בכל סיבוב של הלולאה. הראשון הוא השם (`[0]`), השני המקצוע (`[1]`), והשלישי הציון (`[2]`).

בסיסי נתונים

יש לקרוא היטב ולשים לב לכללים בהקדמה , כולל תרשים מבנה הנתונים (DSD - Data Structure Diagram) ותוכן הטבלאות. מבנה שאלון הבחינה כמו בשנה שעברה

הנחיות ודגשים :

1. מבנה בסיס הנתונים:
   - בסיס הנתונים יכיל 2–3 טבלאות ראשיות ועוד מספר טבלאות עזר קטנות.
   - היות ושמות השדות בכל הטבלאות בבסיס הנתונים הם ייחודיים ,(Unique) וכדי להקל על ההבנה של השאילתות וההשלמות, יופיעו שמות השדות ללא שם הטבלה לפניהם.
2. מפתחות וקשרים בין טבלאות:
   - הבחנה בין מפתח ראשי Primary Key (PK) - שייתכן ויורכב ממספר שדות לבין מפתח זר(FK) Foreign Key .
   - סוגי קשרים בין טבלאות:
     - אחד לאחד(1:1) One to One
     - אחד לרבים(1:N) One to Many
     - רבים לרבים(M:N) Many to Many — יש צורך בטבלאות ביניים / טבלאות מקשרות.
3. סוגי השאלות במבחן:
   - זיהוי תוצאה :איזו טבלה תתקבל כתוצאה מביצוע שאילתה מסוימת?
   - השלמת ערכים :אילו ערכים מספריים או טקסטואליים השאילתה תחזיר?
   - תנאים מורכבים: ביטויים לוגיים, LIKE, Between
     שימוש בכינויים synonym – alias (AS)
   - ניתוח פלט :הצגת טבלה המציגה את הרשומות של שאילתה מסוימת; יש להבין מהי השאילתה שמציגה פלט זה.
   - השלמת פקודות :השלמת פקודות בשאילתות נתונות. *חשוב לציין: יש לכתוב את הפקודות באנגלית בלבד, בצורה ברורה, קריאה ומובנת .*
   - עדכון ומחיקה :פעולות UPDATE / DELETE,לעדכון או מחיקה של ערך שדות בטבלה.
4. פונקציות הקיבוץ :
   - COUNT, SUM, AVG, MIN, MAX .
   - <u>בתנאי קיבוץ באמצעות</u>: WHERE , HAVING
5. שאילתות מקוננות כגון:

SELECT *column_name*

FROM Student WHERE

column_name NOT IN (SELECT ….)

6 .שאילתות מסוג

SELECT DISTINCT *column1, column2, ...*
FROM *table_name*

WHERE …..

לשים דגש על קריטריונים מורכבים עם : LIKE "%XX%" , AND, OR, BETWEEN

7.כתיבת משפט SELECT שיכול להכיל בתוכו שילוב של שמות טבלאות ושאילתות.

<u>לתרגל את הפקודות הבאות :</u>

```
-- Update a table
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;

-- Add a column to a table
ALTER TABLE table_name
ADD column_name datatype;

-- Drop a column from a table
ALTER TABLE table_name
DROP COLUMN column_name;

-- Modify a column's data type
ALTER TABLE table_name
MODIFY COLUMN column_name datatype;

-- Delete rows from a table
DELETE FROM table_name
WHERE condition;

-- Insert rows into a table
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);

-- Drop a table
DROP TABLE table_name;
```